

System Overview

Problem statement

When disaster strikes, internet and cell networks are frequently down, so there is no way to know whether a household is safe, in need, or unreachable unless a person physically finds them and reports back — and the people relied on to do that are disaster victims themselves, often unable to reach everyone who needs them.

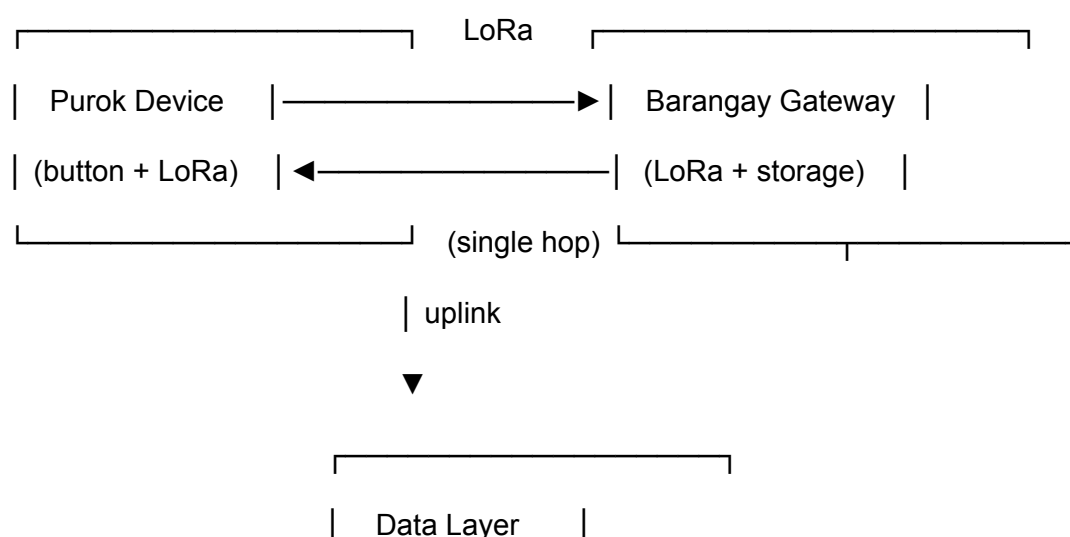
Thesis

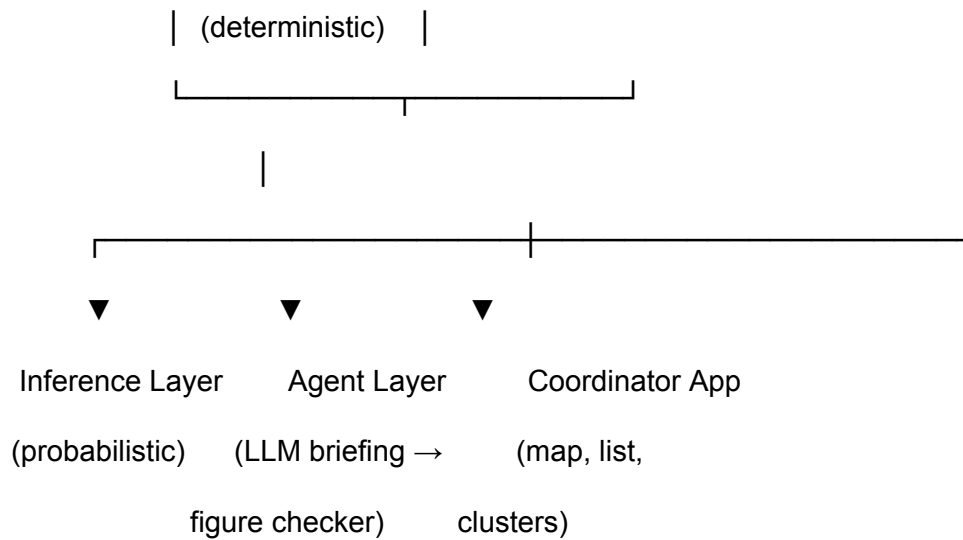
A purok's status should not depend on a person reaching it. A permanently installed device at a known community point reports status directly; a network of gateways moves that signal out without relying on cell infrastructure; an inference layer turns sparse signal into an accurate picture of where help is needed and where nothing has been heard from.

Household vs. purok

- The problem is household-level: nobody knows a specific household's status.
- The device is purok-level: cost and privacy make individual tracking impractical to deploy.
- What this buys: the search narrows from an entire barangay to one purok.
- What it doesn't buy: a person still has to reach that purok to find the specific household.

Architecture at a glance





Components

Component	Role
Purok device	Button-based reporter, one per purok, permanently installed at a known community point
Barangay gateway	Receives LoRa from purok devices, stores, uplinks when possible
Data Layer	Deterministic — storage, identity mapping, fixed rules
Inference Layer	Probabilistic — silence scoring, clustering, severity weighting, anomaly detection
Agent Layer	LLM — briefing synthesis, tool-grounded
Figure Checker	Deterministic — confirms every number in a briefing traces to a tool result

Coordinator App Map, list, clusters, briefing

Purok device and barangay gateway run on the same hardware platform; role is set by firmware, not hardware — see [01_hardware_design.md](#).

Data Layer and Inference Layer commonly run on the same server. The split that matters is logical, not physical:

Given the same input, is there exactly one correct answer — or is this a best guess from incomplete evidence?

One correct answer → Data Layer. Best guess from partial evidence → Inference Layer.

Purok leader

Each device is registered to a purok leader — the recognized point of contact for that purok, accountable for its placement and upkeep. The leader is not a gatekeeper: the device sits at a shared community point, and any purok resident can press it. This keeps reporting from depending on one person being reachable.

Core design principles

1. **Purok device is purok identity.** Fixed at registration.
2. **Deterministic acts, probabilistic proposes.** Rules execute. Inference suggests. A human confirms anything consequential.
3. **Every number is traceable.** Any figure shown to a coordinator resolves to the records behind it.
4. **Silence is signal.** A known purok with no data is a state to investigate, not an absence of information.
5. **No ranking of who receives aid.** A handful of buttons isn't enough signal to justify that decision, and a ranked list gets followed rather than questioned. The system classifies conditions and shows patterns; a person decides.
6. **Coverage gaps are stated, not hidden.** A gateway with no uplink means "unknown," not "stable."

Document index

File

Contents

00_system_overview.md	This document
01_hardware_design.md	Purok device, gateway node, encoding, power
02_data_architecture.md	Entities, schemas, deterministic/probabilistic split
03_agent_design.md	Tool suite, agent, figure checker, guardrails
04_implementation_plan.md	Build phases, what's in and out of scope
05_qa.md	Anticipated questions and answers

Hardware Design

Platform

Purok device and gateway run on the same board type — an ESP32 + LoRa integrated board (e.g. Heltec WiFi LoRa 32, TTGO LoRa32) — flashed with different firmware. One board to source, one bring-up process.

Role is set by firmware:

Purok device firmware

Gateway firmware

LoRa	TX only	RX, listening
Buttons	5–6, wired in	None
Storage	None needed	microSD, buffers until uplink
Power posture	Deep sleep between presses/heartbeat	Less constrained — fixed, attended location

LoRa is used because it needs no access point or cell tower to reach the gateway — direct radio, point to point — and its low power draw is what makes months of battery operation realistic.

Future hardware direction

This build uses an off-the-shelf ESP32 + LoRa board because it's fast to prototype with and available locally this week. A fielded version would move to a **purpose-built module** — MCU and LoRa radio only, nothing the reporting device doesn't use. Lower cost per unit, lower power draw, smaller attack surface.

The gateway is a different case: its job is to gain connectivity, not shed it. A production gateway would likely add a cellular modem rather than relying on WiFi-when-available.

1. Purok device

Build

- The board above, running device firmware
- 5–6 physical buttons, labeled
- Battery (18650 or similar) + optional solar trickle charge
- Weatherproof enclosure

Button semantics

Labeled buttons, not memorized combinations. Recall fails under acute stress; recognition doesn't.

Button	Label (Bisaya)	Meaning
1	TABANG	Distress, general
2	TUBIG	Water need
3	TAMBAL	Medicine need
4	PAGKAON	Food need
5	LUWAS	Nothing urgent right now

LUWAS matters most operationally — it's what keeps ordinary silence from reading as concern. A purok that presses it periodically is different from one that's gone quiet with no signal either way.

Gesture modifiers

Intensity is encoded in *how* a button is pressed, not in which combination is pressed — one thing to remember per press, not a lookup table.

Gesture	Meaning
Single press	Standard report
Two buttons together	Combined need

Packet format

Target payload, well under LoRa's ~250-byte cap:

Field	Size	Notes
<code>device_id</code>	2 bytes	Up to 65,535 devices
<code>seq_num</code>	2 bytes	Monotonic counter, per-device
<code>msg_type</code>	1 byte	0 = event, 1 = heartbeat
<code>button_code</code>	1 byte	Bitmask, one bit per button
<code>press_type</code>	1 byte	0 = single, 1 = hold, 2 = double
<code>timestamp</code>	4 bytes	Unix epoch, device clock

		e	
		s	
batter	1		
y_p		b	
ct		yt	
		e	
checks	1-2		CRC
um		b	
		yt	
		e	
		s	

~12–14 bytes total. Large margin under the payload cap even with LoRa framing overhead.

Dedup key: (**device_id**, **seq_num**), **not a UUID**. A monotonic counter is simpler and cheaper on constrained hardware, and is standard practice for LoRa end devices. This pair is what prevents a retransmitted or duplicate-received packet from being counted twice.

Heartbeat

Periodic keepalive (target: every 4–6 hours) independent of button presses. Establishes the difference between "device is alive and silent" and "device is dead" — required input to the silence-inference tool (see [03_agent_design.md](#)).

Power

Design targets — **to be validated on first firmware bring-up, not assumed:**

- Deep sleep between interrupts: low μA range (chip-dependent)
- LoRa transmit burst: tens of mA, sub-second duration
- Expected duty cycle: dominated by sleep; wake only on button interrupt or heartbeat timer

A single 18650 cell with light solar assist should sustain multi-month operation at this duty cycle. Confirm against the actual board's datasheet before committing to a battery size.

Registration

Registered to a purok, not an individual household. Installed at a shared community point — a chapel, a waiting shed, wherever the purok already gathers — and bound at that time:

- device_id → purok record

- (name, barangay, purok leader, baseline household/vulnerability
- counts from the barangay roster)

The purok leader is the named point of contact — accountable for the device's upkeep and placement — but the device itself sits in a shared, accessible spot. Any purok resident can press it.

Honest limits

- No pre-disaster warning window for earthquakes — device must be a **permanent fixture**, not something distributed as a storm approaches.
 - Distribution and registration is a real logistics project, not a firmware task.
-

2. Gateway

Role

Receives LoRa transmissions from purok devices in range. Stores locally. Forwards upward whenever any uplink path becomes available.

Build

- The same board as the purok device, running gateway firmware
- Local storage (microSD)
- Uplink options, attempted in order of preference:
 1. Cellular/WiFi internet, if available
 2. Serial/USB to a nearby laptop or coordinator device
 3. Manual carry — SD card physically transported

Topology

Star, not mesh. Every purok device talks directly to its assigned gateway. No device-to-device relay, no routing, no density requirement between devices.

Coverage

Realistic LoRa range in hilly, obstructed terrain is meaningfully less than open line-of-sight figures. A barangay will likely need multiple gateways, and some puroks may fall outside any gateway's range.

This is stated, not hidden. Gateway assignment at registration determines expected coverage; a coverage map should show unreachable zones explicitly rather than let them read as "no news."

3. Deployment notes

- Frequency band: 868/915MHz region, per Philippine allocation — confirm before ordering hardware.
- Never power a LoRa module without its antenna attached.
- Gateway placement should prioritize elevation and line-of-sight to the puroks it's assigned to serve.
- One device per purok, not per household, changes the cost picture substantially: a barangay with 15–20 puroks needs 15–20 devices, not hundreds — a low five-figure peso cost for full barangay coverage, phased to highest-risk puroks first if needed.
-

Data Architecture

Entities

Purok

The unit the system tracks. Bound to a device at registration.

Field	Notes
<code>id</code>	
<code>device_id</code>	Fixed at registration
<code>name, barangay</code>	
<code>purok_leader_name,</code> <code>purok_leader_contact</code>	Registered point of contact
<code>baseline_household_count</code>	From barangay roster

<code>baseline_vulnerable_count</code>	Aggregate — elderly, pwd, infant households, from roster
<code>status</code>	unknown / attention / stable
<code>active_needs[]</code>	Need types pressed and not yet resolved — TABANG / TUBIG / TAMBAL / PAGKAON
<code>severity,</code> <code>severity_reasons[]</code>	Current, computed
<code>last_heartbeat_at,</code> <code>last_event_at</code>	
<code>battery_pct</code>	From most recent heartbeat
<code>gateway_id</code>	Assigned gateway

One device maps to exactly one purok. No identity matching or reconciliation is needed — the mapping is fixed at registration, not inferred from a report.

Event

A single transmission from a device. Immutable.

Field	Notes
<code>device_id, seq_num</code>	Composite dedup key
<code>msg_type</code>	event / heartbeat

button_code,
press_type

captured_at Device clock

received_at Gateway/server clock

gateway_id Which gateway received it

rss_i, snr Signal quality — feeds coverage
mapping

Two timestamps exist because a batch synced late shouldn't look like a reporting delay — that would corrupt the silence signals the Inference Layer depends on.

Gateway

Field	Notes
id, barangay, coordinates	
last_uplink_at	
status	online / no_recent_uplink

AuditEvent

actor_id, action, target_type, target_id, timestamp — every read, not only every write.

Deterministic vs. probabilistic

Every operation falls into exactly one column. State this explicitly wherever the system is presented — it's the direct answer to "what if the AI is wrong."

Data Layer — deterministic, executes	Inference Layer — probabilistic, proposes
<code>device_id</code> → <code>purok</code> mapping	Silence interpretation
<code>(device_id, seq_num)</code> idempotent ingestion	Cluster significance across puroks
Time-based severity escalation	Severity pattern-weighting
Access scoping	Anomaly flagging
Audit logging	Briefing synthesis (LLM)
Retention / purge	
Coverage computation	
Figure check on briefing output	

The rule: the probabilistic layer never takes an action. It proposes to the deterministic layer or to a human.

Key deterministic operations

Idempotent ingestion

on event arrival:

if (device_id, seq_num) exists in db → discard, ack

else → insert, update purok.last_event_at

Severity escalation (time component)

hours_since = now - purok.last_event_at

if "TUBIG" in purok.active_needs and hours_since > 48 → escalate to high

if baseline_vulnerable_count > 0 and hours_since > 24 → escalate to high

if last_press_type == "hold" and hours_since > 6 with no follow-up → escalate

Pure function of stored state and clock. Same input, same output, always.

Coverage

purok.coverage_status =

known_unreachable if no gateway in range at registration

no_recent_uplink if assigned gateway.last_uplink_at > threshold

reachable otherwise

no_recent_uplink must never be conflated with **stable**. A silent gateway means unknown, not confirmed.

Access scoping

Role	Sees
Field verifier	Puroks in their assigned area
Barangay coordinator	Their own barangay

Municipal coordinator Aggregates across
 barangays

No role has a standing view of every purok across all barangays.

Audit Every read and write of purok data writes an AuditEvent. Append-only.

Retention

- Event and heartbeat data: purged 90 days after the response period closes
- Purok/device mapping and leader contact: retained as long-term infrastructure

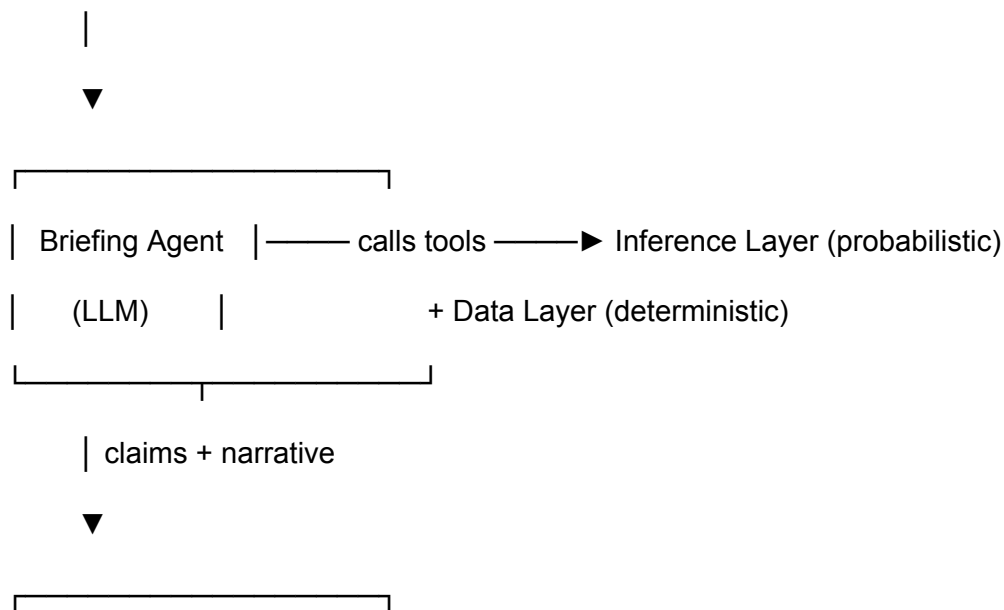
Probabilistic operations

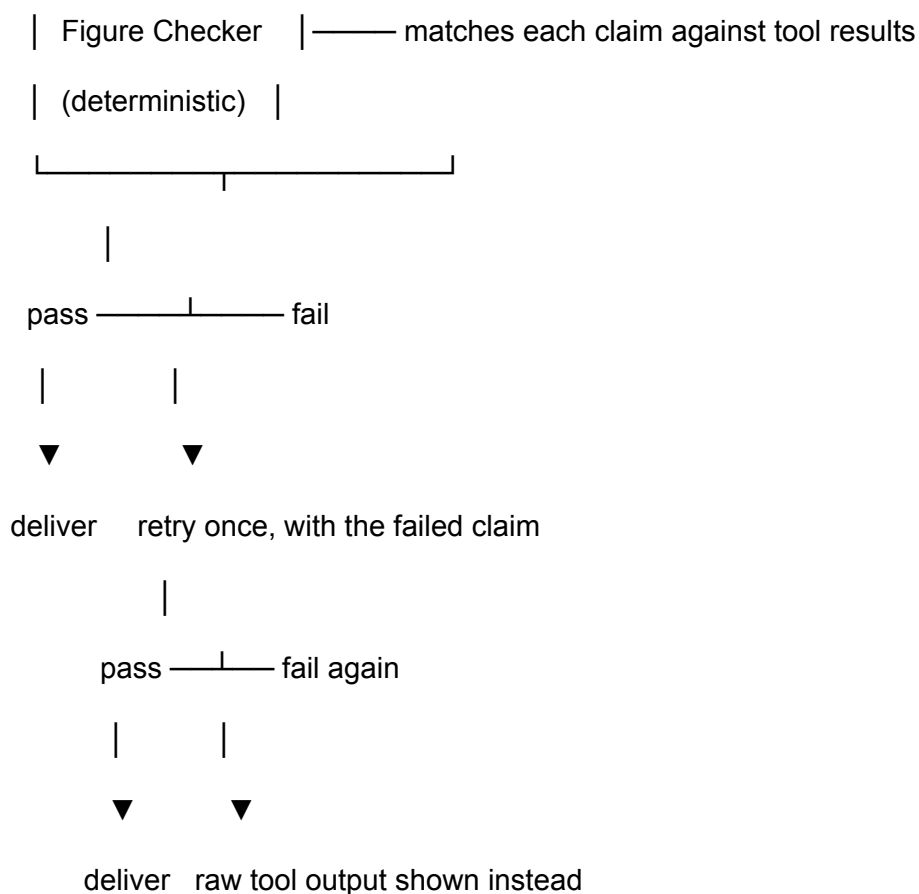
Implemented as a non-LLM statistical/rule-weighted inference engine — not an LLM call.
Detailed in [03_agent_design.md](#) as the tool layer the agent queries.

Agent Design

Architecture

Coordinator query / scheduled trigger / event trigger





Agent

	Type	Function
Briefing Agent	LLM — probabilistic	Selects tools, outputs structured claims plus a narrative built from them

Only one agent. It's the one part of this layer that reasons — everything else it depends on is a fixed rule or a lookup.

Output check

Type	Function
------	----------

Figure Checker	Rule-based — deterministic	Confirms every claim and every number in the narrative traces to a real tool result
----------------	----------------------------	---

How it works:

- The Briefing Agent's output has two parts from one generation: a list of claims (each tagged with which tool and field it came from) and the narrative text built from them
- Per claim: look up its `source_tool` + `source_field` in this turn's actual tool results, confirm the value matches exactly
- On the narrative: extract every number, confirm each appears somewhere in the claims list
- Any mismatch, or any number with no backing claim → fails

On failure: one retry, telling the Briefing Agent exactly which claim was wrong and what the tool actually returned — a much smaller task than generating from scratch. Fails a second time → raw tool output shown instead. No further retries.

Backing services

Tools don't hold logic themselves. Each one reads from exactly one of these.

	Type	Supplies
Data Layer	Deterministic	Purok/device mapping, access rules, coverage status
Inference Layer	Probabilistic	Silence scores, cluster detection, severity weighting, anomaly flags

How Inference Layer scoring works — no LLM involved: A table of rules, each with a condition, a weight, and a fixed reason phrase. One generic loop checks every rule against the current numbers and sums what fires.

RULES = [

```
{ condition: "hours_since_heartbeat > 6", weight: 30, reason: "no heartbeat in over 6h" },
{ condition: "hours_since_heartbeat > 12", weight: 30, reason: "no heartbeat in over 12h" },
{ condition: "neighbors_silent", weight: 20, reason: "neighboring puroks also quiet" },
```

```
{ condition: "last_press == held_TABANG", weight: 40, reason: "last press was a held TABANG" },  
]
```

for rule in RULES:

```
    if evaluate(rule.condition, purok_state):  
  
        score += rule.weight  
  
        reasons.append(rule.reason)
```

Adding a rule is a new row, not new code. The reason text is exactly what fired — no gap between what the score says and why.

Tool suite

get_unaccounted_puroks(barangay?) — Inference Layer

- **purok_id, purok_name**
- **hours_since_contact** — time since last heartbeat or press
- **silence_score** — 0–100, how concerning the silence is
- **reason** — e.g. "no heartbeat, neighboring puroks also quiet"

get_active_clusters(time_window?) — Inference Layer

- **cluster_id**
- **need_type** — TABANG / TUBIG / TAMBAL / PAGKAON
- **puroks[]** — which puroks are in this cluster
- **window_minutes** — how tightly packed in time
- **confidence** — 0–100

get_high_severity(barangay?) — Inference Layer

- **purok_id, purok_name**
- **severity_level** — low / medium / high
- **reasons[]** — e.g. "no water reported, 52h", "held TABANG, 3 times"

get_anomalies() — Inference Layer

- **device_id, purok_name**
- **anomaly_type** — e.g. "press pattern inconsistent with this device's own history"

- `flagged_at`

`get_recent_activity(time_window)` — Data Layer

- `total_events` — how many presses and heartbeats arrived in the window
- `by_need_type` — count per TABANG / TUBIG / TAMBAL / PAGKAON / LUWAS
- `puroks_reporting` — how many distinct puroks sent anything

`get_purok(device_id)` — Data Layer

- Full purok record — name, barangay, purok leader, baseline counts, current status, severity
- `event_history[ ]` — every past press and heartbeat from this device

`get_coverage(barangay?)` — Data Layer

- `gateway_id, status` — online / no_recent_uplink
- `puroks_affected[ ]` — puroks that report through this gateway
- `hours_since_uplink`

Guardrails

- No mutating tools
- Every figure and purok reference links to its source record
- No priority ranking — classifications and clusters only
- A purok behind a gateway with no recent uplink is shown as unknown, never stable
- LLM unavailable → raw tool output shown instead

Trigger modes

Mode	When
On-demand	Coordinator asks a question
Scheduled	Every 4 hours
Event-triggered	Cluster or severity threshold crossed

Example output

Coordinator: "What's changed since 2pm?"

Response:

"18 new events since 14:00. Three puroks in this barangay — Purok 3, Purok 5, Purok 7 — pressed TUBIG within 45 minutes of each other. Reads as a barangay-wide water problem, not three separate issues. Purok 9 has not sent a heartbeat in 9 hours; the last press before that was a held TABANG. Gateway 2 has not received an uplink in 6h — Puroks 4, 6, and 8 report through it and currently show as unknown, not stable."

[Purok 3] [Purok 5] [Purok 7] [Purok 9] [Gateway 2] → tappable to source records

Implementation Plan

What's being built

- A button device at a shared spot in each purok
- A receiving unit at the barangay
- A dashboard for coordinators, with a plain-language summary of what's changed

Included

- Button device + wireless signal
- Barangay receiving unit
- Coordinator dashboard
- Pattern detection — turns scattered presses into a clear picture
- Plain-language summary of what's changed
- Access control and record-keeping

Not included

- Resident phone app
 - Text messaging
 - Voice reporting
 - Disaster prediction
 - Public map
 - Full data export
-

Phase 1 — Signal reaches a receiving unit

Button pressed → device sends signal → barangay unit receives it

- Built and proven first
- Cannot be skipped or simplified later
- Everything else depends on this working

Phase 2 — Signal becomes information

Signal received → matched to a purok → checked against time since

last report → given a status: normal / needs attention / not heard from

- Protect: knowing when a purok goes quiet
- Can wait: anything more detailed

Phase 3 — Seeing the whole picture

Multiple puroks report the same need close together in time

→ recognized as one shared problem

- Hardest piece to build
- If not finished: individual purok statuses still work on their own
- Can be added later without breaking anything

Phase 4 — A summary a person can read

Raw data → short written summary of what changed and what needs attention → every number checkable against the real record

- If not finished: raw data is still fully usable
- This is a convenience, not a requirement

Phase 5 — Keeping data safe

- Who can see what
 - A record of who looked at what, and when
 - A clear limit on how long data is kept
 - Built alongside everything else, not left for last
-

What it must do when shown to someone

Real button pressed

- real wireless signal, no cable between devices
- status appears on dashboard within seconds
- second press elsewhere shows as a shared pattern
- plain-language summary describes what happened, checkable against the real data

This sequence is the whole point. Everything else supports it.

Anticipated Questions

Problem and approach

Why purok-level, not household-level? Isn't that the actual problem?

- The problem is household-level. The device is purok-level — cost and deployability, not a redefinition of the problem.
- What it buys: the search narrows from a whole barangay to one purok.
- What it doesn't buy: a person still has to reach that purok to find the specific household.

Isn't this just another disaster app?

- The common patterns are a public hazard map, an SOS button, or a chatbot.
- The coordinator dashboard does include a map — but it's role-scoped, for coordinators only, not a public map anyone can open.
- Built around a specific, documented failure: most displaced people during a disaster often shelter outside evacuation centres, where relief systems don't look.

Why not just use phones?

- Assumes a working, charged phone and a network to report over — often false in the exact moment this matters.
- A permanent community device removes that dependency entirely.

Hardware and connectivity

Why LoRa and not WiFi, SMS, or Bluetooth mesh?

- WiFi needs an access point; there isn't one at a rural community point.
- SMS needs a working cell network, which is the thing that's down.
- Mesh needs device density to relay hop to hop; sparse, hilly terrain doesn't have it.
- LoRa is direct radio, point to point, and low-power enough to run on battery for months.

What happens if the gateway goes down?

- Puroks that report through it show as unknown, never as stable or safe.
- No signal either way is not treated as good news.

What's the real range and coverage?

- Realistic LoRa range in obstructed, hilly terrain is meaningfully less than open line-of-sight figures.
- A barangay will likely need multiple gateways; some puroks may sit outside any gateway's range.
- This is shown on the coverage map, not hidden.

Why build custom hardware instead of using what already exists?

- This build uses an off-the-shelf ESP32 + LoRa board — fast to prototype, available locally.
- A fielded version would move to a purpose-built module: MCU and radio only, nothing unused. Lower cost, lower power draw, smaller attack surface.

AI and the agent layer

What if the AI is wrong?

- The Briefing Agent's draft is checked against the actual tool results before anything reaches a coordinator.
- A number with no matching tool result fails the check; the agent gets one retry with the specific error, then falls back to raw data if it fails again.

Why only one agent, not several?

- Coverage, clustering, and severity aren't independent — a purok that's anomalous, in a cluster, and behind a downed gateway is one situation, not three.
- Splitting that into agents that don't share context would force re-synthesizing a correlation that was artificially cut apart.
- A second agent is warranted by a second, genuinely independent data source — e.g. structural sensors added later — not by more judgment types on the same data.

Is the verification step itself an LLM? What if it's also wrong?

- No. Checking whether a number matches a tool result has exactly one correct answer — a fixed rule handles it, not a second model.
- A rule can't hallucinate a false pass.

How does severity or silence get scored without an LLM?

- A table of rules, each with a condition, a weight, and a fixed reason phrase. One generic loop sums what fires.
- Same inputs always produce the same score. The reason shown is exactly what fired — no gap between the answer and why it was given.

Why doesn't the system rank who gets help first?

- A handful of buttons isn't enough signal to justify that decision.
- A ranked list gets followed rather than questioned, which removes the human judgment the system depends on.
- The system classifies conditions and shows patterns; a person decides.

Data and misuse

What data does this actually hold?

- Per purok: a device-to-purok mapping, a registered point of contact, and aggregate baseline counts from the barangay roster.
- Per event: a need category, a gesture, and a timestamp — no names, no health details.

What stops this from being used for patronage?

- No role has a standing view of the full roster.
- Every read and write is logged.
- This is deterrence and traceability, not prevention — worth stating plainly rather than overclaiming.

Is consent really possible during a disaster?

- Full informed consent isn't realistic in the moment. What's achievable: awareness at registration, before the disaster, and refusal being a valid outcome.

Scope and limitations

What happens if someone presses the button falsely, or it's tampered with?

- Placement is deliberately public and visible — harder to tamper with unnoticed than something kept in one household.
- Anomaly detection flags a device behaving inconsistently with its own history, for a human to review.

Is this scalable beyond one barangay?

- More puroks and barangays scale the data layer and inference engine, not the number of agents.
- More concurrent coordinators scale the orchestrator horizontally. Neither requires architectural changes.

What does deployment actually cost?

- One device per purok, not per household — a barangay with 15–20 puroks needs 15–20 devices, not hundreds.
- Low five-figure peso range for full barangay coverage, phased to highest-risk puroks first if needed.

Why isn't there a resident phone app?

- Explicitly out of scope. The device removes the dependency on a working, charged phone — adding a phone app back in would reintroduce it for anyone without one.
-